Arquitectura Modular Integral para la Detección y Mitigación de Alucinaciones en LLMs: Diseño, Implementación y Estrategias de Escalabilidad

1. Fundamentos de la Ingeniería de Fiabilidad en Modelos de Lenguaje

La integración operativa de los Modelos de Lenguaje Grande (LLMs) en ecosistemas empresariales y aplicaciones críticas se enfrenta a una barrera fundamental: la propensión estocástica a la generación de contenido no factual, fenómeno comúnmente denominado "alucinación".[1] A medida que las organizaciones transitan de pruebas de concepto aisladas a sistemas de producción robustos, la necesidad de arquitecturas que no solo generen texto, sino que también cuantifiquen la incertidumbre epistémica y verifiquen la facticidad atómica, se ha convertido en un requisito no funcional de primer orden.[3]

El desafío técnico reside en la naturaleza de caja negra de los modelos comerciales (OpenAI, Gemini) y la variabilidad de recursos de los modelos locales (Ollama). Una solución monolítica es insuficiente para abordar la complejidad de orquestar múltiples proveedores de inferencia, ejecutar algoritmos de detección intensivos en cómputo como la Entropía Semántica [2] y gestionar flujos de recuperación correctiva (CRAG).[5] Este informe define una arquitectura modular, desacoplada y agnóstica al proveedor, diseñada para operar en un entorno híbrido donde la privacidad de los datos (modelos locales) convive con la capacidad de razonamiento superior (modelos de frontera).

La arquitectura propuesta se fundamenta en la premisa de que la detección de alucinaciones no es un paso único, sino un proceso multicapa que involucra la introspección del modelo (incertidumbre interna) y la verificación externa (RAG y búsqueda web). Al utilizar patrones de diseño de software avanzados —como Factory Pattern para la instanciación de LLMs y grafos de estado para la orquestación lógica— garantizamos que el sistema sea mantenible y extensible ante la rápida evolución del estado del arte en Inteligencia Artificial Generativa.

---

2. Paradigma Arquitectónico: El Sistema Híbrido Modular

Para satisfacer los requisitos de flexibilidad (integración de Ollama, OpenRouter, Gemini,

OpenAI) y robustez (Self-RAG/CRAG), la arquitectura se aleja de los scripts lineales tradicionales para adoptar un diseño orientado a servicios y eventos. El sistema se estructura en capas concéntricas de responsabilidad, donde la capa de inferencia es abstracta y la capa de lógica de negocio es declarativa.

2.1 Descomposición de Componentes Principales

La arquitectura se divide en cuatro dominios funcionales principales, cada uno encapsulado para permitir actualizaciones independientes sin afectar la integridad del sistema global.

Dominio Funcional	Responsabilidad Principal	Tecnologías Clave	Patrón de Diseño
Capa de Abstracción de Inferencia (IAL)	Normalizar la comunicación con diversos proveedores de LLM (Local/Nube).	LiteLLM, Ollama, OpenAI SDK	Abstract Factory / Adapter
Motor de Orquestación Cognitiva (COE)	Gestionar el flujo de razonamiento, recuperación y decisión (Self-RAG/CRAG).	LangChain, LangGraph	State Machine (Grafo de Estado)
Servicios de Evaluación y Detección (EDS)	Ejecutar algoritmos de verificación de facticidad y cálculo de incertidumbre.	PyTorch, HuggingFace (NLI), FActScore	Worker/Queue (Celery/Redis)
Interfaz de Observabilidad (FEO)	Visualizar métricas de confianza y permitir la interacción humana en el bucle.	React, D3.js/Recharts, WebSockets	Component-Based UI

Esta separación permite, por ejemplo, cambiar el modelo de embeddings en el COE sin alterar la interfaz de visualización en el FEO, o sustituir GPT-4 por Claude 3.5 en la IAL sin reescribir la lógica de detección de alucinaciones en el EDS.[6]

2.2 Flujo de Datos y Ciclo de Vida de la Solicitud

El flujo de datos se diseña para manejar la latencia inherente a los procesos de verificación. A diferencia de un chatbot estándar, donde la prioridad es el tiempo al primer token (TTFT), aquí la prioridad es la integridad de la respuesta.

1. **Recepción y Enrutamiento:** La solicitud del usuario llega a la API Gateway (FastAPI). El sistema identifica el perfil de configuración (e.g., "¿Usar modelo local Llama-3 para privacidad o GPT-4 para razonamiento complejo?").
2. **Inferencia Inicial y Recuperación:** El COE inicia el grafo de LangGraph. Dependiendo de la estrategia (Self-RAG o CRAG), se recuperan documentos y se genera una respuesta preliminar.[8]
3. **Bifurcación de Evaluación (Forking):**
   - **Ruta Rápida (Stream):** La respuesta preliminar se envía al frontend vía WebSocket para UX inmediata.
   - **Ruta Profunda (Async):** Se envían tareas a la cola de mensajes (Redis) para los trabajadores de EDS. Estos calculan la Entropía Semántica (generando múltiples muestras paralelas) y ejecutan FActScore (verificando hechos atómicos).[10]
4. **Sincronización y Mitigación:** Si el EDS detecta alta incertidumbre o baja puntuación factual, el COE puede interrumpir el flujo, activar una reescritura de la consulta (Query Transformation) o inyectar una advertencia en tiempo real al usuario.
5. **Presentación:** El FEO recibe los metadatos de evaluación y actualiza la visualización, superponiendo mapas de calor de incertidumbre sobre el texto generado.[11]

---

3. Capa de Inferencia: Abstracción Universal con LiteLLM y Factory Pattern

La exigencia de integrar modelos locales (Ollama), APIs directas (Gemini, OpenAI) y enrutadores (OpenRouter) presenta un desafío de fragmentación de interfaces. Cada proveedor tiene firmas de API ligeramente diferentes, gestión de errores propia y formatos de respuesta distintos. Para resolver esto, implementamos una capa de abstracción robusta utilizando **LiteLLM** como normalizador y el patrón **Factory** para la inyección de dependencias.[12]

3.1 Diseño del Adaptador Universal de Modelos

El objetivo es tratar a cualquier modelo, ya sea un Llama-3-70b ejecutándose en una GPU A100 local o un GPT-4o en la nube, como una entidad intercambiable. LiteLLM facilita esto al traducir las llamadas al formato estándar de OpenAI Chat Completion, pero la arquitectura debe gestionar la configuración y el ciclo de vida de estas conexiones.

Utilizamos una clase base abstracta LLMService que define el contrato para cualquier interacción de generación. Las implementaciones concretas (e.g., OllamaService, OpenRouterService) son instanciadas por una LLMFactory basada en la configuración del

entorno o la preferencia del usuario por solicitud.[6]

Especificación Técnica de la Factoría:

La factoría debe manejar no solo la creación del cliente, sino también la estrategia de reintento y la gestión de *rate limits*. LiteLLM proporciona decoradores para manejar excepciones comunes (RateLimitError, ServiceUnavailableError) de manera unificada, lo cual es crítico cuando se mezcla tráfico hacia APIs estables (OpenAI) con APIs de proveedores más volátiles o servidores locales que pueden saturarse.[14]

El siguiente esquema lógico ilustra la selección dinámica:

- Si el usuario solicita provider="local", la factoría instancia un cliente apuntando a http://localhost:11434 (puerto estándar de Ollama).
- Si solicita provider="openrouter", apunta a https://openrouter.ai/api/v1 inyectando la OPENROUTER_API_KEY.
- La lógica de negocio (LangGraph) recibe el objeto cliente agnóstico y simplemente llama a acompletion().

3.2 Estrategias de Gestión para Modelos Locales (Ollama)

La integración de **Ollama** introduce restricciones físicas que no existen en las APIs de nube: la memoria VRAM y la concurrencia. Si el sistema intenta ejecutar un cálculo de Entropía Semántica (que requiere generar 10 muestras) en paralelo con una solicitud de chat entrante en una sola GPU, el servidor local puede colapsar o degradar severamente el rendimiento.

Para mitigar esto, la arquitectura implementa un **Semáforo de Concurrencia Distribuida** (utilizando Redis) específico para las colas de tareas que apuntan a modelos locales.

- **Cola Prioritaria:** Las solicitudes de chat interactivas tienen prioridad alta.
- **Cola de Fondo:** Los cálculos de FActScore y Entropía para modelos locales se encolan con prioridad baja y se procesan secuencialmente o en lotes pequeños para no saturar la VRAM.
- **Keep-Alive:** Se configura Ollama con un keep_alive prolongado para evitar la descarga del modelo de la memoria entre ráfagas de solicitudes de evaluación, reduciendo la latencia de carga del modelo (cold start).

3.3 Integración de OpenRouter y Modelos de Frontera

OpenRouter actúa como un *middleware* de mercado, permitiendo acceder a modelos como Claude 3 Opus o Llama 3 70B Instruct alojados en diversos proveedores (DeepInfra, Fireworks, etc.).[15] La arquitectura aprovecha esto para la **mitigación de costos y la especialización**:

- **Enrutamiento por Complejidad:** El sistema puede configurarse para usar un modelo local (Ollama/Phi-3) para la evaluación inicial de relevancia de documentos (nodo

grade_documents en Self-RAG) debido a su bajo costo y velocidad, y reservar modelos de frontera (via OpenRouter/GPT-4) únicamente para la generación final o la resolución de conflictos complejos de alucinación.
- **Failover Automático:** LiteLLM permite configurar listas de modelos de respaldo. Si la API de Gemini falla, el sistema conmuta transparentemente a OpenAI o a un modelo de alto rendimiento en OpenRouter, garantizando alta disponibilidad.[16]

---

4. Motor de Detección I: Entropía Semántica (Semantic Entropy)

La **Entropía Semántica (SE)** representa un avance significativo sobre las métricas de incertidumbre tradicionales (como la perplejidad o la entropía de tokens bruda) que a menudo fallan en tareas de generación libre porque confunden la diversidad léxica con la incertidumbre semántica.[2] En esta arquitectura, SE es el mecanismo principal para detectar cuando el modelo "no sabe" la respuesta pero está confabulando una contestación plausible.

4.1 Fundamento Matemático y Algorítmico

La entropía tradicional $H(Y|x)$ mide la incertidumbre sobre la secuencia específica de palabras. Sin embargo, la frase "Roma es la capital de Italia" y "La capital de Italia es Roma" son semánticamente idénticas pero léxicamente distintas. SE agrupa estas generaciones en clases de equivalencia semántica.

El algoritmo implementado sigue el método de **Clustering por Implicación Bidireccional (Bi-directional Entailment Clustering)** propuesto por Kuhn et al. y Farquhar et al..[17]

Proceso de Cálculo:

1. **Muestreo Estocástico:** Dado un contexto x, se generan M secuencias de respuesta $s_1, \dots, s_M$ utilizando el LLM con una temperatura elevada (e.g., $T=1.0$) para explorar el espacio de probabilidad del modelo.[19]
2. **Clustering Semántico:**
   - Se define una relación de equivalencia $s_i \equiv s_j$ si $s_i \implies s_j$ y $s_j \implies s_i$ (implicación mutua).
   - Se utiliza un algoritmo codicioso (Greedy Clustering) para agrupar las respuestas. Se toma la primera respuesta s_1 como centroide del primer clúster. Para cada s_i subsiguiente, se verifica si implica y es implicada por el centroide de los clústeres existentes. Si hay coincidencia, se une al clúster; si no, forma uno nuevo.[20]
3. **Estimación de Entropía:**
   - La probabilidad de un significado (clúster) c se aproxima por la fracción de muestras que contiene: $p(c|x) \approx \frac{|c|}{M}$.
   - La Entropía Semántica se calcula sumando sobre los clústeres: $SE(x) \approx -

\sum_{c} p(c|x) \log p(c|x)$.

4.2 Implementación Modular del Evaluador de Implicación (NLI)

La arquitectura desacopla el "Verificador de Implicación" para permitir optimizaciones de rendimiento y costo. El sistema soporta dos modos de operación configurables en el backend:

1. **Modo NLI Dedicado (Local/GPU):** Utiliza un modelo Cross-Encoder optimizado como deberta-v3-large-mnli ejecutándose localmente.[19]
   - *Ventaja:* Latencia extremadamente baja y costo cero por inferencia. Ideal para altos volúmenes.
   - *Implementación:* Se carga el modelo en un servicio separado (contenedor Docker) que expone una API interna gRPC o HTTP rápida. Las comparaciones se realizan por lotes (batching) para maximizar el uso de la GPU.
2. **Modo LLM-as-a-Judge (API):** Utiliza un LLM ligero (e.g., gpt-4o-mini o llama-3-8b-instruct) con un prompt específico ("¿La frase A implica que la frase B es verdadera?").
   - *Ventaja:* No requiere gestión de infraestructura de ML local; mejor capacidad de razonamiento en dominios complejos donde DeBERTa podría fallar.
   - *Desventaja:* Mayor latencia y costo acumulativo.

La arquitectura implementa una caché de resultados de implicación (Memoization) basada en el hash de las oraciones normalizadas, reduciendo drásticamente los cálculos redundantes cuando las muestras son idénticas.[23]

4.3 Estrategia de Muestreo y Latencia

Generar 5-10 respuestas completas introduce una latencia significativa. Para mitigar el impacto en la experiencia del usuario (UX), el cálculo de SE se realiza de manera **asíncrona y desacoplada**:

- El usuario recibe la primera respuesta (Generación Greedy o con temperatura baja) inmediatamente.
- En segundo plano, el sistema dispara el proceso de muestreo y cálculo.
- Una vez completado, el frontend recibe una notificación (WebSocket) y actualiza la interfaz mostrando un indicador de "Confianza Semántica" o un mapa de calor, alertando al usuario si la respuesta generada inicialmente podría ser una alucinación basada en la alta entropía detectada a posteriori.[24]

---

5. Motor de Detección II: FActScore (Puntuación de Precisión Factual)

Mientras que la Entropía Semántica evalúa la consistencia interna, **FActScore** evalúa la veracidad externa descomponiendo una generación larga en una serie de hechos atómicos y

verificando cada uno contra una base de conocimiento fiable (e.g., Wikipedia, documentos corporativos indexados).[26]

5.1 Pipeline de Descomposición y Verificación

La implementación de FActScore en esta arquitectura se diseña como un pipeline de tres etapas orquestado por Celery, permitiendo escalabilidad horizontal.

1. **Extracción de Hechos Atómicos:**
   - El texto generado se envía a un modelo (puede ser un modelo local Llama-3 vía Ollama para ahorrar costos) con la instrucción de segmentar la respuesta en oraciones declarativas simples que contengan un solo hecho verificable.
   - *Ejemplo:* "Barack Obama nació en Hawái en 1961" $\rightarrow$.
2. **Recuperación de Evidencia (Context Retrieval):**
   - Cada hecho atómico se utiliza como consulta para el sistema de recuperación (Vector Store). Aquí la arquitectura reutiliza el índice vectorial construido para el módulo RAG, asegurando consistencia entre la generación y la evaluación. Se recuperan los k pasajes más relevantes.[28]
3. **Verificación de Hechos (Fact-Checking):**
   - Un modelo "Juez" toma el hecho atómico y los pasajes recuperados y determina una etiqueta binaria: Soportado o No Soportado.
   - El puntaje final es el porcentaje de hechos soportados sobre el total de hechos extraídos.

5.2 Optimización con Modelos Locales e Inst-LLAMA

El paper original de FActScore propone el uso de **Inst-LLAMA**, una versión de Llama ajustada para seguir instrucciones, como una alternativa eficiente a GPT-4 para la evaluación.[26]

Nuestra arquitectura integra soporte nativo para usar modelos locales de Ollama en este paso crítico. Dado que FActScore requiere una llamada al LLM por cada hecho atómico (lo que puede sumar docenas de llamadas para un texto largo), usar APIs comerciales sería prohibitivo.

- **Configuración Local:** Se define un perfil de evaluación en el backend que enruta las tareas de descomposición y juicio a una instancia dedicada de Ollama ejecutando llama3 o una versión cuantizada de Inst-LLAMA.
- **Abstención y Desconocimiento:** Se implementa lógica para manejar casos donde el modelo no tiene suficiente información, clasificándolos distintamente de las alucinaciones directas ("Irrelevante" vs "Falso").[28]

5.3 Integración de Datos y Personalización

El sistema permite configurar la "Fuente de Verdad". Por defecto, se utiliza el índice vectorial conectado (que puede contener la base de conocimiento de la empresa). Sin embargo, para

conocimientos generales, se puede configurar el pipeline para consultar un volcado de Wikipedia local o realizar búsquedas web en tiempo real (integrándose con el módulo CRAG) para verificar hechos de actualidad.[29]

---

6. Motor de Mitigación: Flujos de Trabajo RAG con LangGraph

La detección es el diagnóstico; la mitigación es la cura. La arquitectura utiliza **LangGraph** para implementar arquitecturas cognitivas que van más allá de las cadenas lineales, introduciendo ciclos de retroalimentación y toma de decisiones condicionales.[30]

6.1 Implementación de Self-RAG (Self-Reflective RAG)

Self-RAG introduce "tokens de reflexión" (o nodos de evaluación en nuestra implementación de grafo) para autoevaluar la calidad de la recuperación y la generación. El grafo se define con los siguientes estados y transiciones [30]:

- **Nodo Retrieve:** Consulta el almacén vectorial.
- **Nodo Grade Documents:** Un LLM ligero evalúa la relevancia de cada fragmento recuperado con la consulta del usuario. Filtra el ruido antes de la generación.
  - *Optimización:* Este paso es ideal para modelos locales pequeños (e.g., Phi-3 o Llama-3-8B) por su alta velocidad.
- **Nodo Generate:** Genera la respuesta utilizando solo los documentos validados.
- **Nodo Check Hallucination (Post-Generación):** Evalúa si la generación se adhiere a los documentos (Faithfulness). Si falla, el grafo hace un bucle hacia atrás (Generate) con una instrucción de corrección.
- **Nodo Check Utility:** Evalúa si la respuesta es útil para la consulta original. Si no, activa el nodo Transform Query.

Mantenimiento Futuro: Al definir estos pasos como nodos independientes en un grafo de LangGraph, es trivial insertar nuevos pasos de validación (e.g., "Check Toxicity") o cambiar la lógica de enrutamiento sin reescribir todo el flujo de control.

6.2 Implementación de CRAG (Corrective RAG)

CRAG se centra en la robustez de la recuperación. Si el conocimiento interno es insuficiente, el sistema debe saber buscar fuera.[33]

- **Evaluador de Recuperación:** Si el nodo Grade Documents descarta todos los documentos recuperados (calidad baja) o si la incertidumbre es ambigua, se activa una bandera de estado web_search: True.
- **Nodo Web Search:** Este nodo integra herramientas de búsqueda externa (API de Tavily o SerpApi). La arquitectura modular permite inyectar estas herramientas como

dependencias.
- **Nodo Transform Query:** Antes de buscar en la web, un LLM reescribe la consulta del usuario para optimizarla para motores de búsqueda (e.g., convirtiendo una pregunta vaga en keywords precisas).

El uso de **LangGraph** permite visualizar estas decisiones en tiempo real en el frontend, mostrando al usuario cuándo el sistema decidió "salir" a internet para verificar una alucinación potencial.

---

7. Ingeniería Backend: FastAPI, Asincronía y Patrones de Escalabilidad

El backend actúa como el director de orquesta, gestionando la concurrencia y la comunicación entre los servicios de inferencia y los clientes.

7.1 FastAPI y Gestión de Dependencias

Se utiliza **FastAPI** por su rendimiento asíncrono y su sistema de inyección de dependencias, crucial para el diseño modular.

Inyección de Dependencias para Configuración Dinámica:
Se implementa un sistema donde la configuración del modelo y las herramientas se inyectan en tiempo de ejecución. Esto permite que una sola instancia de la API sirva a múltiples tenants con configuraciones distintas (uno usando OpenAI, otro usando Ollama local).34

Python

```python
# Ejemplo conceptual del patrón de inyección
async def get_graph_workflow(
    llm_service: Annotated,
    vector_store: Annotated
):
    # Construye el grafo Self-RAG inyectando las implementaciones concretas
    return SelfRAGGraph(llm=llm_service, retriever=vector_store)
```

7.2 Gestión de Tareas en Segundo Plano (Background Tasks vs. Celery)

Dado que los cálculos de Entropía Semántica y FActScore son intensivos, no deben bloquear el hilo principal de la respuesta HTTP.[36]

- **FastAPI BackgroundTasks:** Se utilizan para tareas ligeras y rápidas, como el registro de logs de auditoría o actualizaciones simples de metadatos en la base de datos tras una respuesta exitosa.
- **Celery con Redis:** Se utiliza para la orquestación pesada. Los cálculos de alucinación se envían a colas de Celery. Esto permite escalar los "workers" de evaluación independientemente de los nodos de la API. Si la carga de detección aumenta, se pueden añadir más contenedores de workers sin tocar la API de servicio.[10]
    - *Patrón Async Request-Reply:* El cliente recibe un task_id inmediatamente al enviar un mensaje. El frontend sondea (polling) o escucha un WebSocket suscrito a ese task_id para recibir las métricas de alucinación cuando estén listas, desacoplando la generación de texto (rápida) de la verificación de hechos (lenta).[38]

---

8. Ingeniería Frontend: Observabilidad y Visualización de Incertidumbre

La interfaz de usuario no es meramente un chat; es una herramienta de observabilidad para la IA. Construida con **React**, debe gestionar flujos de datos complejos y visualizaciones ricas.

8.1 Visualización de Incertidumbre con Mapas de Calor

Para comunicar la Entropía Semántica al usuario, implementamos componentes de **resaltado de texto (Text Highlighting)** dinámicos.

- **Componente UncertaintyHighlighter:** Recibe el texto generado y un array de puntuaciones de entropía a nivel de token o frase. Utiliza una escala de color secuencial (e.g., de transparente a rojo intenso) para indicar visualmente las zonas de baja confianza.[11]
- **Interacción:** Al hacer hover/clic sobre una frase resaltada, un componente Tooltip o Popover muestra detalles granulares:
    - Puntuación exacta de entropía.
    - Hechos atómicos identificados por FActScore y su estado de verificación.
    - Fragmentos de documentos recuperados que contradicen o apoyan la afirmación (trazabilidad de evidencia).
- **Bibliotecas:** Se recomienda el uso de react-text-annotate o componentes personalizados sobre span con estilos en línea calculados (backgroundColor: interpolateOranges(entropy)) para un control preciso y ligero.[40]

8.2 Trazabilidad del Grafo en Tiempo Real

Para generar confianza en el sistema Self-RAG/CRAG, el usuario debe ver "cómo piensa" el modelo.

- **Visualización de Pasos:** Un componente de "Stepper" o diagrama de flujo simplificado

muestra el estado actual del grafo de LangGraph (e.g., "Recuperando...", "Evaluando...", "Reescribiendo consulta...").
- **Streaming de WebSockets:** El backend de FastAPI envía eventos de cambio de estado del grafo a través de WebSockets. React consume estos eventos para actualizar la UI instantáneamente, reduciendo la percepción de latencia en flujos complejos que involucran múltiples pasos de razonamiento.[42]

---

9. Infraestructura, Despliegue y Mantenimiento Futuro

9.1 Contenerización Estratégica

El despliegue se orquesta mediante **Docker Compose** (dev) y **Kubernetes** (prod) para gestionar la heterogeneidad de los servicios.

- **Contenedor API:** Ejecuta FastAPI + LiteLLM. Escala basado en CPU/Requests.
- **Contenedor Worker (GPU):** Ejecuta los modelos locales de NLI (Entropía) y Ollama (si se autohospeda). Requiere acceso al runtime de NVIDIA (--gpus all).
- **Contenedor Vector DB:** ChromaDB o Weaviate. Debe tener persistencia de volumen para el índice.
- **Contenedor Broker:** Redis para la gestión de colas de Celery y caché de respuestas.

9.2 Estrategias de Mantenimiento y Evolución

- **Evaluación Continua (CI/CD for AI):** Se debe implementar un pipeline que ejecute periódicamente un "Golden Dataset" a través de la arquitectura. FActScore y Entropía Semántica sirven aquí como métricas de regresión: si una actualización del modelo o del prompt aumenta la alucinación en el dataset de prueba, el pipeline falla.[43]
- **Versioning de Grafos:** LangGraph permite versionar la lógica de los grafos. La arquitectura debe guardar qué versión del grafo generó cada respuesta en la base de datos de trazas, permitiendo auditorías forenses sobre la lógica de decisión utilizada en el pasado.
- **Agnosticismo de Proveedor:** Al usar LiteLLM y el patrón Factory, el sistema es resistente a cambios en el mercado (e.g., subida de precios de OpenAI). Cambiar de proveedor es una operación de configuración, no de reescritura de código.[13]

9.3 Tabla Resumen de Tecnologías y Roles

Componente	Tecnología Seleccionada	Justificación Técnica
Backend Framework	FastAPI (Python)	Async nativo, Pydantic, ecosistema AI fuerte.

Adaptador LLM	LiteLLM	Interfaz unificada para >100 modelos, manejo de errores.
Orquestación	LangGraph	Soporte para ciclos, estado y persistencia de memoria.
Cola de Tareas	Celery + Redis	Manejo robusto de tareas pesadas (detección) desacopladas.
Modelos Locales	Ollama	Fácil despliegue, gestión de VRAM, API REST simple.
Vector Store	ChromaDB / Weaviate	Soporte híbrido, escalabilidad, integración LangChain.
Frontend	React + TypeScript	Ecosistema de visualización rico, tipado seguro.
Visualización	D3.js / Custom CSS	Control total sobre mapas de calor y anotaciones de texto.

Esta arquitectura representa un enfoque integral y profesional para el problema de las alucinaciones. No solo integra las herramientas más avanzadas disponibles (LangGraph, FActScore), sino que las organiza en una estructura mantenible que prioriza la modularidad, la observabilidad y la flexibilidad operativa, preparada para evolucionar junto con la rápida innovación en el campo de la Inteligencia Artificial Generativa.

Obras citadas

1. [2510.06265] Large Language Models Hallucination: A Comprehensive Survey - arXiv, fecha de acceso: enero 14, 2026, https://arxiv.org/abs/2510.06265
2. Beyond Semantic Entropy: Boosting LLM Uncertainty Quantification with Pairwise Semantic Similarity - ACL Anthology, fecha de acceso: enero 14, 2026, https://aclanthology.org/2025.findings-acl.234.pdf
3. A Survey on Hallucination in Large Language Models: Principles, Taxonomy, Challenges, and Open Questions - arXiv, fecha de acceso: enero 14, 2026, https://arxiv.org/abs/2311.05232

4. A Survey of Uncertainty Estimation Methods on Large Language Models - ACL Anthology, fecha de acceso: enero 14, 2026, https://aclanthology.org/2025.findings-acl.1101.pdf
5. langgraph/examples/rag/langgraph_crag.ipynb at main · langchain ..., fecha de acceso: enero 14, 2026, https://github.com/langchain-ai/langgraph/blob/main/examples/rag/langgraph_crag.ipynb
6. How Design Patterns Saved My FastAPI Project | by Juan D Khusuma - Medium, fecha de acceso: enero 14, 2026, https://medium.com/@juan.d.khusuma/how-design-patterns-saved-my-fastapi-project-665a1160c02f
7. Organizing AI Applications: Lessons from traditional software architecture - DEV Community, fecha de acceso: enero 14, 2026, https://dev.to/redis/organizing-ai-applications-lessons-from-traditional-software-architecture-mpe
8. Corrective RAG (CRAG) Implementation With LangGraph - DataCamp, fecha de acceso: enero 14, 2026, https://www.datacamp.com/tutorial/corrective-rag-crag
9. This project implements a Self-RAG (Self-Refining Retrieval-Augmented Generation) system using LangChain, LangGraph, Groq LLMs, and Chroma VectorStore. - GitHub, fecha de acceso: enero 14, 2026, https://github.com/Gihan007/Self-RAG-Systems
10. What's the difference between FastAPI background tasks and Celery tasks? - Stack Overflow, fecha de acceso: enero 14, 2026, https://stackoverflow.com/questions/74508774/whats-the-difference-between-fastapi-background-tasks-and-celery-tasks
11. Visualizing Uncertainty in Translation Tasks: An Evaluation of LLM Performance and Confidence Metrics - arXiv, fecha de acceso: enero 14, 2026, https://arxiv.org/html/2501.17187v1
12. A gentle introduction to LiteLLM. Unify LLM APIs across OpenAI, Ollama... | by Tituslhy | MITB For All | Medium, fecha de acceso: enero 14, 2026, https://medium.com/mitb-for-all/a-gentle-introduction-to-litellm-649d48a0c2c7
13. LiteLLM - Getting Started | liteLLM, fecha de acceso: enero 14, 2026, https://docs.litellm.ai/
14. LiteLLM - Getting Started, fecha de acceso: enero 14, 2026, https://docs.litellm.ai/docs/
15. OpenRouter vs LiteLLM: Enterprise guide to reducing LLM expenses - Xenoss, fecha de acceso: enero 14, 2026, https://xenoss.io/blog/openrouter-vs-litellm
16. Best Practices for Production | liteLLM, fecha de acceso: enero 14, 2026, https://docs.litellm.ai/docs/proxy/prod
17. Semantic uncertainty in advanced decoding methods for LLM generation - arXiv, fecha de acceso: enero 14, 2026, https://arxiv.org/html/2506.17296v1
18. Detecting hallucinations in large language models using semantic entropy - PMC - NIH, fecha de acceso: enero 14, 2026, https://pmc.ncbi.nlm.nih.gov/articles/PMC11186750/
19. SelfCheckGPT: Zero-Resource Black-Box Hallucination Detection for Generative

Large Language Models - GitHub, fecha de acceso: enero 14, 2026, https://github.com/potsawee/selfcheckgpt
20. Algorithm outline for bidirectional entailment clustering Given a set... - ResearchGate, fecha de acceso: enero 14, 2026, https://www.researchgate.net/figure/Algorithm-outline-for-bidirectional-entailment-clustering-Given-a-set-of-outputs-in_fig1_381550138
21. SEMANTIC ENTROPY PROBES: ROBUST AND CHEAP HALLUCINATION DETECTION IN LLMS - OpenReview, fecha de acceso: enero 14, 2026, https://openreview.net/pdf?id=YQvvJjLWX0
22. Releases · potsawee/selfcheckgpt - GitHub, fecha de acceso: enero 14, 2026, https://github.com/potsawee/selfcheckgpt/releases
23. Propaganda via AI? A Study on Semantic Backdoors in Large Language Models - arXiv, fecha de acceso: enero 14, 2026, https://arxiv.org/pdf/2504.12344
24. Celery and Background Tasks. Using FastAPI with long running tasks | by Hitoruna | Medium, fecha de acceso: enero 14, 2026, https://medium.com/@hitorunajp/celery-and-background-tasks-aebb234cae5d
25. Generating with Confidence: Uncertainty Quantification for Black-box Large Language Models - arXiv, fecha de acceso: enero 14, 2026, https://arxiv.org/pdf/2305.19187
26. FActScore: Fine-grained Atomic Evaluation of Factual Precision in Long Form Text Generation - ACL Anthology, fecha de acceso: enero 14, 2026, https://aclanthology.org/2023.emnlp-main.741/
27. factscore - PyPI, fecha de acceso: enero 14, 2026, https://pypi.org/project/factscore/
28. A package to evaluate factuality of long-form generation. Original implementation of our EMNLP 2023 paper "FActScore - GitHub, fecha de acceso: enero 14, 2026, https://github.com/shmsw25/FActScore
29. An Analysis of Multilingual FActScore - ACL Anthology, fecha de acceso: enero 14, 2026, https://aclanthology.org/2024.emnlp-main.247.pdf
30. langgraph/examples/rag/langgraph_self_rag.ipynb at main - GitHub, fecha de acceso: enero 14, 2026, https://github.com/langchain-ai/langgraph/blob/main/examples/rag/langgraph_self_rag.ipynb
31. langgraph/examples/rag/langgraph_self_rag_local.ipynb at main - GitHub, fecha de acceso: enero 14, 2026, https://github.com/langchain-ai/langgraph/blob/main/examples/rag/langgraph_self_rag_local.ipynb
32. langgraph/examples/rag/langgraph_self_rag.ipynb at main - GitHub, fecha de acceso: enero 14, 2026, https://github.com/langchain-ai/langgraph/blob/main/examples/rag/langgraph_self_rag.ipynb?ref=blog.langchain.dev
33. Corrective RAG (CRAG) - GitHub Pages, fecha de acceso: enero 14, 2026, https://langchain-ai.github.io/langgraph/tutorials/rag/langgraph_crag/
34. Dependencies - FastAPI, fecha de acceso: enero 14, 2026, https://fastapi.tiangolo.com/tutorial/dependencies/

35. How to Implement Dependency Injection in FastAPI - freeCodeCamp, fecha de acceso: enero 14, 2026, https://www.freecodecamp.org/news/how-to-implement-dependency-injection-in-fastapi/
36. LangServe vs FastAPI: Key Differences, Use Cases & Setup Guide - Leanware, fecha de acceso: enero 14, 2026, https://www.leanware.co/insights/langserve-vs-fastapi
37. Recommendations other than celery to send an API processing in background, which would only take 5 mins to process and API usage would be once a month or so. : r/django - Reddit, fecha de acceso: enero 14, 2026, https://www.reddit.com/r/django/comments/zj60wt/recommendations_other_than_celery_to_send_an_api/
38. Asynchronous Request-Reply pattern - Azure Architecture Center | Microsoft Learn, fecha de acceso: enero 14, 2026, https://learn.microsoft.com/en-us/azure/architecture/patterns/async-request-reply
39. How to visualize the attention vector with a text heatmap? - Stack Overflow, fecha de acceso: enero 14, 2026, https://stackoverflow.com/questions/72865289/how-to-visualize-the-attention-vector-with-a-text-heatmap
40. Text annotation component for React - GitHub, fecha de acceso: enero 14, 2026, https://github.com/vlddlv/react-text-annotation
41. Building a Text Annotation Component with React - JonathanMH, fecha de acceso: enero 14, 2026, https://jonathanmh.com/p/building-a-text-annotation-component-with-react/
42. Set Up FastAPI and HTTP - Ray Serve, fecha de acceso: enero 14, 2026, https://docs.ray.io/en/latest/serve/http-guide.html
43. Design and Develop a RAG Solution - Azure Architecture Center | Microsoft Learn, fecha de acceso: enero 14, 2026, https://learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/rag/rag-solution-design-and-evaluation-guide